CAESAR CIPHER ENCRYPTION AND DECRYPTION

PRESENTED BY:

Ishika - 2023BCSE07AED254

Suman Supriya B -2023BCSE07AED172

A.Shaik Shane Rabbani -2023BCSE07AED631

Pragya - 2023BCSE07AED257

INTRODUCTION

What is a Caesar Cipher?

- A substitution cipher that shifts letters in the alphabet by a fixed number.
- One of the simplest and oldest encryption methods.
- Used for basic text encryption and learning cryptographic concepts.

Objective of the Project

- Implement a simple Caesar Cipher encryption and decryption tool.
- Use JavaScript for logic, HTML for structure, and CSS for styling.
- Provide user controls for shift value, alphabet set, and letter case.

Working Mechanism

- Each letter in the text is replaced with a letter ‘n’ positions down the alphabet.
- Wraps around the end of the alphabet.
- Example: If the shift key is 3, ‘A’ becomes ‘D’, ‘B’ becomes ‘E’, etc.
- Decoding reverses the shift operation.

Implementation Details

- **Technologies Used:**
  - HTML (Structure)
  - CSS (Styling)
  - JavaScript (Logic & Functionality)
- **Features:**
  - Encoding & Decoding
  - Customizable shift key
  - Custom alphabet input
  - Letter case options
  - Handling foreign characters

Code Structure

- **HTML:** Form inputs, radio buttons, text areas.
- **CSS:** Styling for a clean and responsive UI.
- **JavaScript:** Core encryption & decryption logic using string manipulation

Code Breakdown - HTML

- Provides the basic structure for user interaction.
- Contains input fields for text and shift value.
- Includes buttons for encryption and decryption

```
<div class="container">
    <label for="text">Enter Text:</label>
    <input type="text" id="text" />
    <label for="shift">Shift Value:</label>
    <input type="number" id="shift" value="3" />
    <div class="buttons">
        <button onclick="encrypt()">Encrypt</button>
        <button onclick="decrypt()">Decrypt</button>
    </div>
    <p id="result"></p>
</div>
```

Code Explanation - CSS

- Styles the elements for better user experience.
- Uses Flexbox to center elements and create a responsive design.

```css
body {
    font-family: Arial, sans-serif;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    background-color: #f4f4f4;
}
.container {
    display: flex;
    flex-direction: column;
    align-items: center;
    background: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
.buttons {
    display: flex;
    gap: 10px;
    margin-top: 10px;
}
```

Code Explanation - JavaScript Logic

- Defines the encryption and decryption logic.
- Uses ASCII values to shift characters correctly.
- Ensures both uppercase and lowercase letters are handled properly

```
function caesarCipher(str, shift, decrypt = false) {
    if (decrypt) shift = -shift;
    return str.split('').map(char => {
        let code = char.charCodeAt(0);
        if (char >= 'A' && char <= 'Z') {
            return String.fromCharCode(((code - 65 + shift + 26) % 26) + 65);
        } else if (char >= 'a' && char <= 'z') {
            return String.fromCharCode(((code - 97 + shift + 26) % 26) + 97);
        }
        return char;
    }).join('');
}

function encrypt() {
    let text = document.getElementById('text').value;
    let shift = parseInt(document.getElementById('shift').value);
    document.getElementById('result').innerText = caesarCipher(text, shift);
}

function decrypt() {
    let text = document.getElementById('text').value;
    let shift = parseInt(document.getElementById('shift').value);
    document.getElementById('result').innerText = caesarCipher(text, shift, true);
}
```

Challenges Faced

- Handling special characters and spaces.
- Maintaining letter cases.
- Managing shift overflow beyond the alphabet

Comparison with Modern Encryption

- **Caesar Cipher**: Simple, easy to break, used for learning purposes.
- **Modern Encryption** (AES, RSA): Secure, used for real-world applications.
- Caesar Cipher **not suitable for serious security needs** but good for basic encryption

Future Enhancements

- Implementing additional ciphers (e.g., Vigenère cipher).
- Adding graphical representation of encryption.
- Creating a mobile-friendly version.

Conclusion

- A simple yet effective project to understand basic cryptography.
- Showcases real-world applications of string manipulation in JavaScript.
- Provides a foundation for learning advanced encryption techniques.